LEARNING UNIT 3 · ACTIVITY 3

Customer Care FAQ Chatbot with RAG

The agent answers only from the brochures — or admits it does not know.

PLATFORM	VECTOR STORE	DURATION
Copilot Studio	Pinecone	90 minutes

Building AI Agents for Work Automation

Tertiary Infotech Academy Pte Ltd · UEN 201200696W

01

Why RAG?

The problem that retrieval solves

SCENARIO

The Customer Care Problem

40 times a day

Cook & Bake Academy runs 20 courses across two campuses. Two people answer the same questions: how much, how long, where, what level.

The answer already exists

It is in the course brochures. Nobody is short of information — somebody has to find the right file, read it, and retype a sentence.

Memory drifts

When the team is busy they answer from memory. Last month someone quoted a fee that was six months out of date.

A language model could answer all of this — if it had ever seen the brochures.

THE NAIVE APPROACH

Why Not Just Paste the Brochures In?

Paste all 20 into the prompt

Works today, with 20 short brochures.

Then the academy adds 40 more courses.

The prompt exceeds what the model can read.

It starts ignoring the middle of long input.

Every question costs the price of 60 brochures.

Retrieve only what is needed

Search for the 3 brochures that resemble the question.

Send only those.

Cost stays flat as the catalogue grows.

The model reads 3 documents carefully instead of 60 badly.

This is RAG.

RAG is not a smarter model. It is a smaller, better-chosen prompt.

Retrieval Augmented Generation

Retrieve the few documents that answer the question, then let the model generate an answer from those and nothing else.

CLEARING THREE MISCONCEPTIONS

What RAG Is Not

Not fine-tuning

No training. No model weights change. You are choosing what goes in the prompt, not rebuilding the model.

Not a database query

There is no exact match. You ask for the nearest meanings, and you always get results — even for a question nothing answers.

Not a guarantee

Retrieval supplies facts. Only the instruction stops the model adding its own. Both halves are required.

If you remember one thing: RAG changes the prompt, not the model.

02

How It Works

Tokens, embeddings, dimensions, similarity

CONCEPT 1 OF 4

Step 0 — Tokenization: How a Model Reads Text

1. **A model never sees letters — it sees tokens**
   Text is split into pieces of roughly ¾ of a word. "sourdough" might be two tokens: "sour" + "dough".

2. **Everything is counted in tokens**
   Context limits, pricing and speed are all measured in tokens — not words, not characters.

3. **A rule of thumb for English**
   1 token ≈ 4 characters ≈ ¾ of a word. One 2,700-character brochure is roughly 700 tokens.

4. **Why this matters for RAG**
   20 brochures ≈ 14,000 tokens in every prompt. Retrieve 3 instead and you send ~2,100 — the same answer for a seventh of the input.

Tokenization is why "just paste everything in" has a ceiling, and why retrieval saves money.

CONCEPT 2 OF 4

Step 1 — Embedding: Turning Text Into Coordinates

1. **Text in, numbers out**
   An embedding model converts a piece of text into a fixed-length list of numbers — a vector.

2. **Position encodes meaning**
   The model places text so that similar meanings land near each other. This is learned, not programmed.

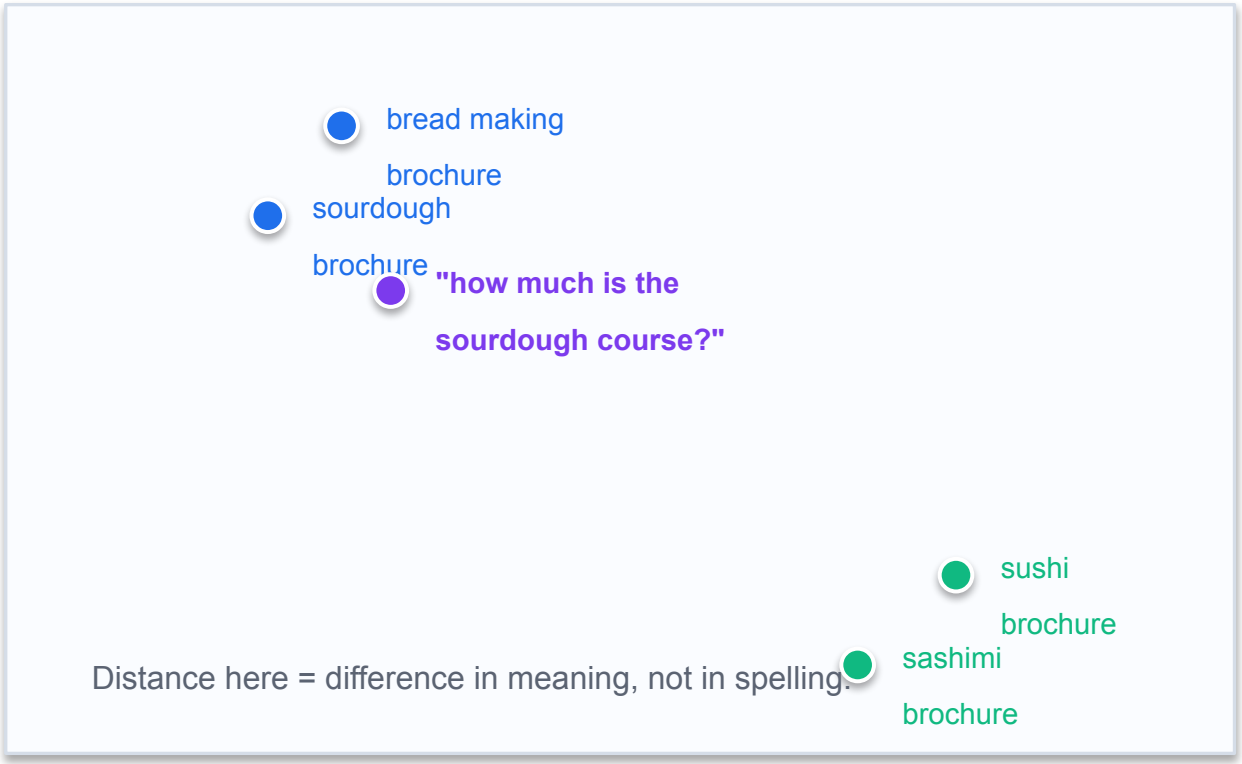
3. **Same model, every time**
   Brochures and questions must be embedded by the SAME model, or the coordinates are not comparable.

4. **In this lab it is invisible**
   Pinecone's hosted model (llama-text-embed-v2) embeds server-side. You send text, not numbers.

An embedding is a position in meaning-space. Nothing more mysterious than that.

CONCEPT 3 OF 4

Semantic Search: Near Means Similar



This is why a customer who misspells “viennoiserie” still finds BAK-102.

What the picture shows

The question lands next to the sourdough brochure and far from the sushi one — even though the question and the brochure share almost no words.

A keyword search would need the exact word. A vector search needs only the meaning.

Real embeddings do this in 1024 dimensions, not 2. You cannot draw that, but the idea is identical: near = similar.

Keyword Search vs Vector Search

Keyword search — matches characters

"viennoiserie" misspelt → zero results.

"french pastry classes" → no match, because the brochure never uses the word "classes".

Finds documents that contain your letters.

Fails silently on synonyms, plurals and typos.

Vector search — matches meaning

"vienoiserie" (misspelt) → still finds BAK-102.

"french pastry classes" → finds BAK-102 French Pastry & Viennoiserie.

Finds documents that mean what you asked.

Always returns something — even for a question nothing answers.

That last line cuts both ways: vector search never says "no results". Guarding against a confident answer built on a poor match is the instruction's job.

CONCEPT 4 OF 4

Step 2 — Dimension: How Many Numbers

1. **The dimension is the length of the vector**
   llama-text-embed-v2 produces 1024 numbers per piece of text. Gemini's gemini-embedding-001 produces 3072.

2. **More dimensions ≠ better**
   It is a different model with a different trade-off between nuance, storage and speed. Not a quality dial.

3. **The index is fixed at one dimension**
   An index built for 1024 accepts 1024-length vectors and nothing else.

4. **The rule you cannot break**
   ingestion model == query model == the index's dimension. All three, or the search is meaningless.

Break rule 4 and you get either a loud error or — far worse — quietly poor results.

FAILURE MODES

The Two Ways Dimension Bites You

Loud failure — you will notice

A 1536-dim query against a 3072-dim index.

Pinecone rejects it with an error naming both numbers.

Annoying, but it stops you immediately.

Fixed in five minutes.

Silent failure — you will not

Both models give 3072 numbers — but they are different models.

The index accepts every vector. No error, ever.

Retrieval just returns slightly wrong brochures, forever.

This is the one that reaches production.

Gemini also needs taskType RETRIEVAL_DOCUMENT when ingesting and RETRIEVAL_QUERY when searching. Both return 3072 numbers, so getting it wrong throws no error at all.

THE SETTING THAT DECIDES QUALITY

Step 3 — Chunking: The Biggest Lever

1. **A chunk is the unit that gets embedded and returned**
   Search does not return “a document”. It returns whichever chunk matched.

2. **Chunk too small: 1,000 characters**
   A 2,700-character brochure becomes 3 fragments. The fragment saying “sourdough” is not the fragment saying “S$680”.

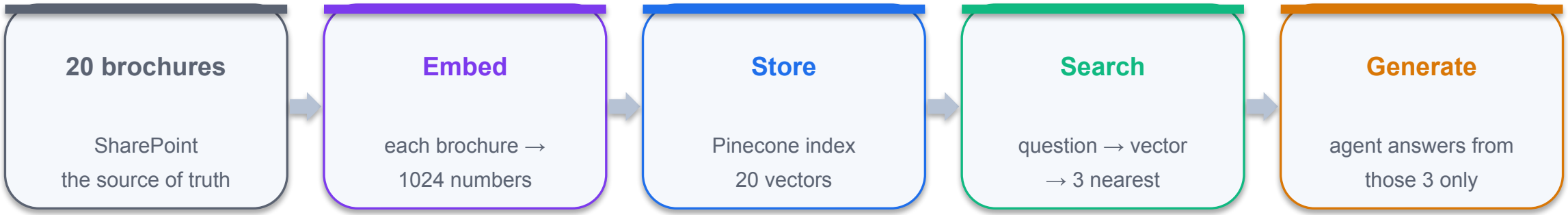
3. **The result is an honest, useless answer**
   Retrieval finds the sourdough fragment. The agent never sees the fee. It tells the customer it does not know the price. No error is raised.

4. **This lab: one brochure = one chunk**
   2,700 characters, well within the model's 2,048-token limit. One search returns the code, the fee, the duration and the campus together.

The rule: a chunk is the smallest piece of text that still answers a question on its own. For a brochure that is the whole brochure. For a 400-page manual it is not.

PUTTING THE FOUR CONCEPTS TOGETHER

The RAG Pipeline, End to End



The first three run ONCE (ingestion). The last two run on EVERY question (retrieval). That split is why RAG is cheap to operate and slow to set up.

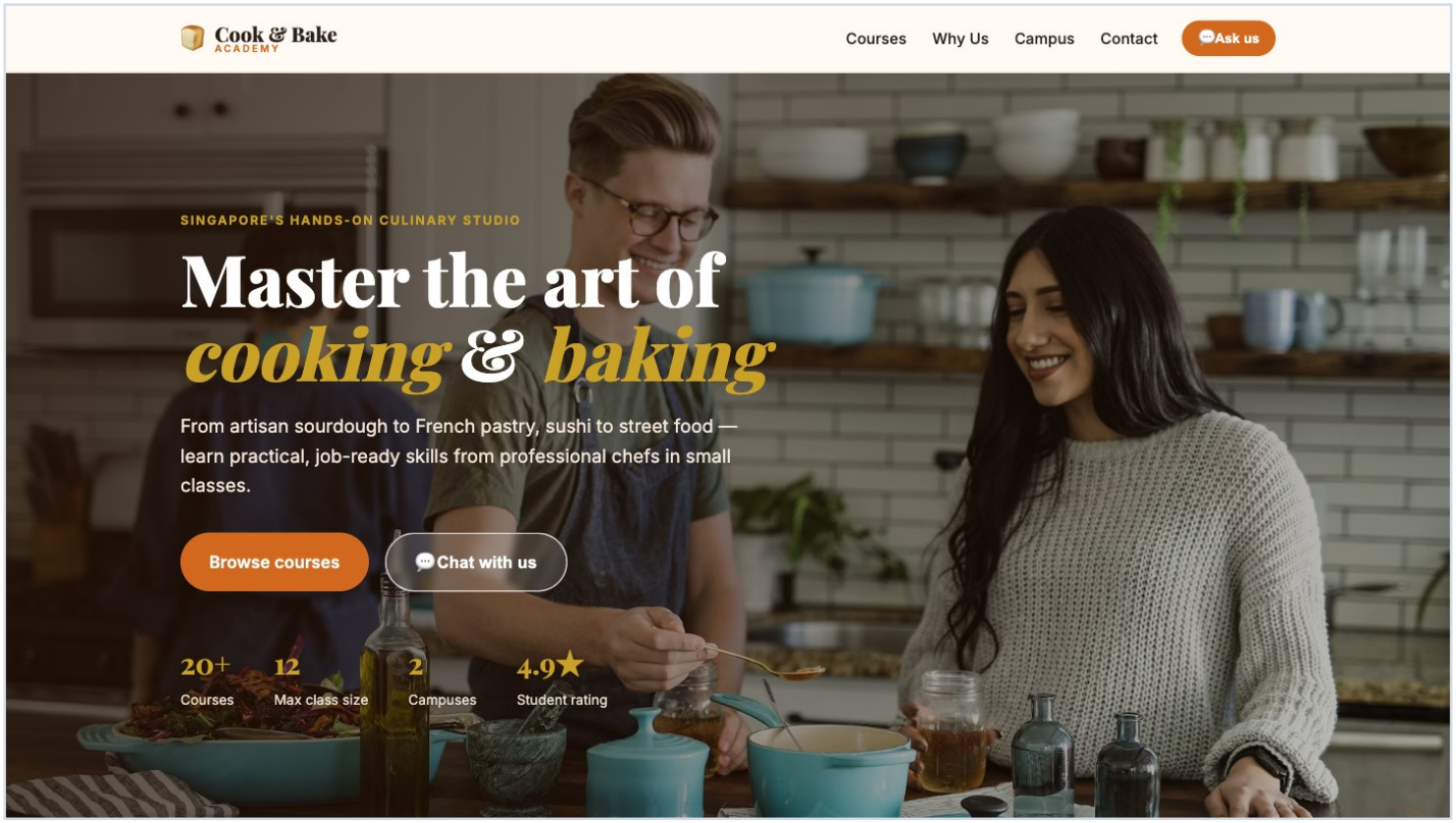
03

The Build

Five nodes, one flow

THE DELIVERABLE

What You Are Building



The academy's website. The 💬 button opens a chat widget that posts to your flow.

One Flow, Five Nodes

Plus Task 1, which ran once before any of this existed: create the index, upload the 20 brochures as text. Notice there is no embeddings node — Pinecone hosts the model.

RUN ONCE, BEFORE ANY FLOW EXISTS

Task 1 — Ingest: Create the Index

1. **POST /indexes/create-for-model**
   One call creates a serverless index with an INTEGRATED embedding model — Pinecone hosts the model, so you upload words, not numbers.

2. **model: llama-text-embed-v2**
   1024 dimensions, 2048-token limit. You did not pick the dimension — it is a property of the model.

3. **field_map: { text: chunk_text }**
   THE embedding configuration. It says: embed whatever arrives in the field called chunk_text.

4. **Get that name wrong and nothing errors**
   Records are stored with no vector at all. Retrieval returns nothing, forever, and no log says why.

python3 ingest_brochures.py — the script is 150 commented lines and is the only place in this lab where the embedding decisions are visible.

AS TEXT, NOT AS VECTORS

Task 1 — Ingest: Upload the Brochures

1. **POST /records/namespaces/__default__/upsert**
   Content-Type: application/x-ndjson — one JSON object per line, no wrapping array, no commas between lines.

2. **One record per brochure, whole**
   {"_id":"BAK-101", "chunk_text":"…2,740 characters…", "course_code":"BAK-101"}

3. **chunk_text is embedded — everything else is metadata**
   source and course_code come back with each hit and can be filtered on. They are never embedded.

4. **Verify before you build the flow**
   20 vectors in the index, and “How much is the sourdough course?” returns BAK-101 at 0.53. If that fails, no flow will save you.

~685 tokens per brochure — comfortably inside the model's 2,048-token limit, so nothing is truncated and nothing is split.

THE BIG DIFFERENCE FROM N8N

Where Did the Embedding Model Go?

n8n Activity 3 — you attach it TWICE

Vector Store node + Embeddings node.

Retrieval tool + Embeddings node.

You pick the model: gemini-embedding-001, 3072 dims.

taskType: passage to write, query to read.

Get either wrong → no error, quietly worse.

This lab — it lives inside the index

The flow has ONE HTTP node. No embeddings node anywhere.

You send the sentence. Pinecone embeds it server-side.

The model was fixed when the index was created.

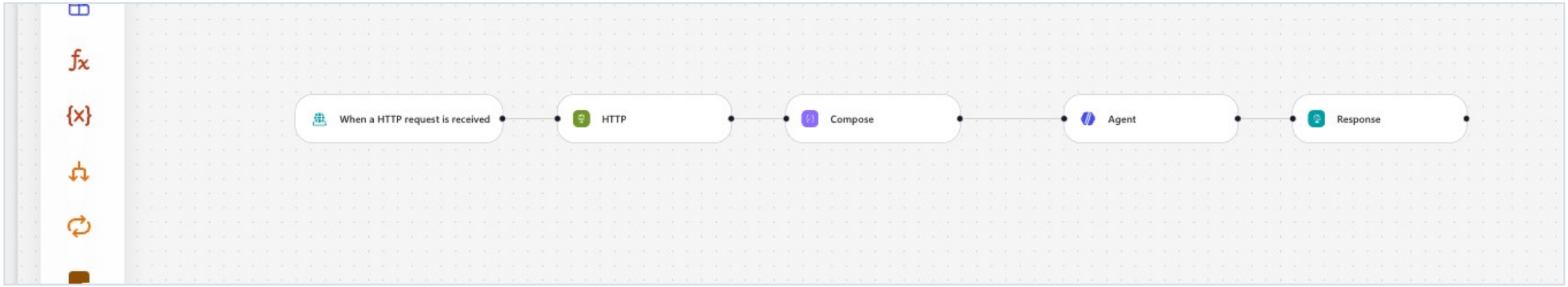
passage-vs-query is handled by the index.

Nothing to mismatch — there is nothing to choose.

You bought: a whole class of silent failure, removed. You paid: you cannot change the model, the dimension, or use a domain-specific embedding without rebuilding the index.

WHAT IT LOOKS LIKE WHEN FINISHED

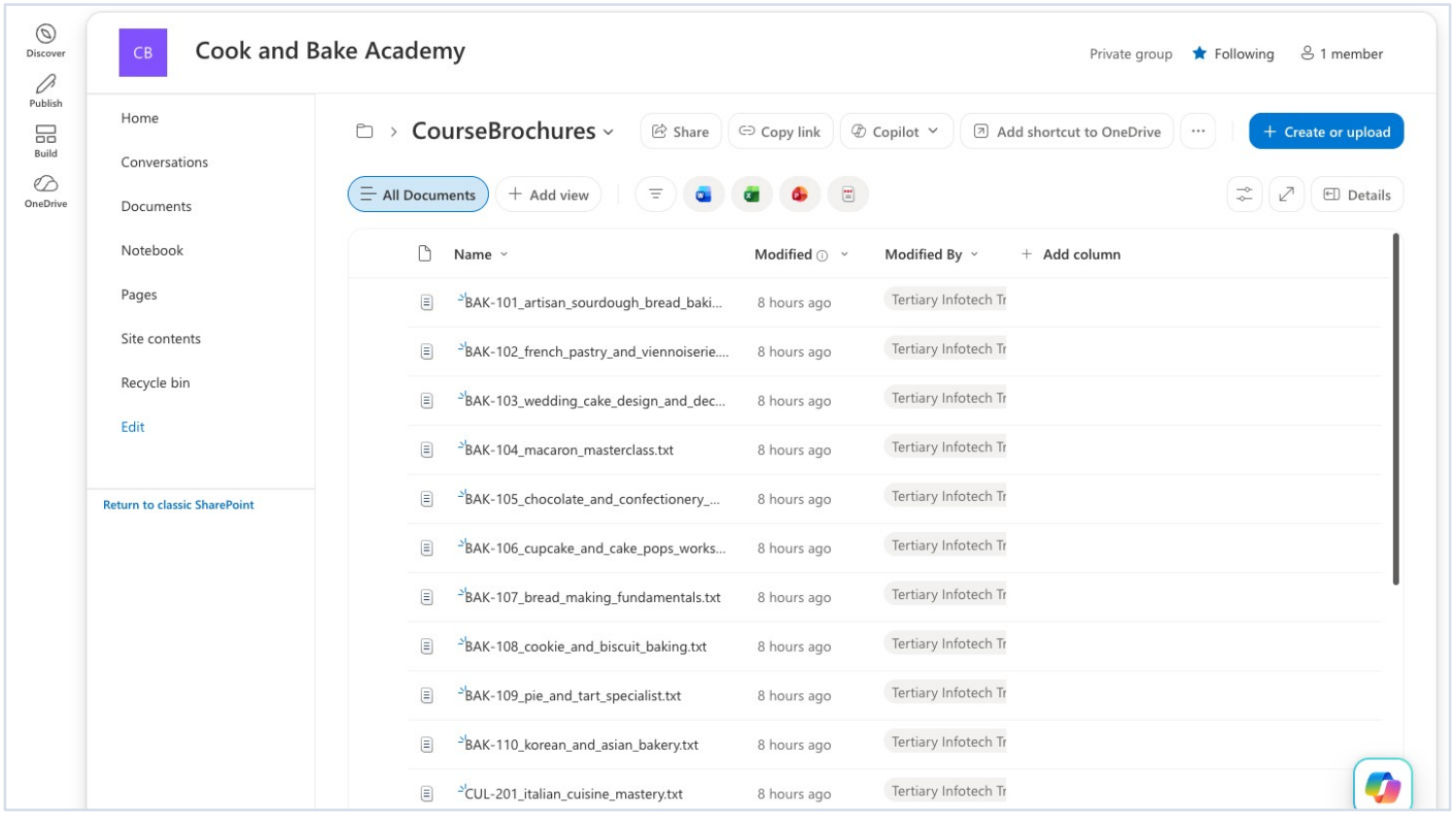
The Flow on the Canvas



Published, five nodes, no “Needs setup” badges.

WHERE THE TRUTH LIVES

Task 0 — The Brochures in SharePoint

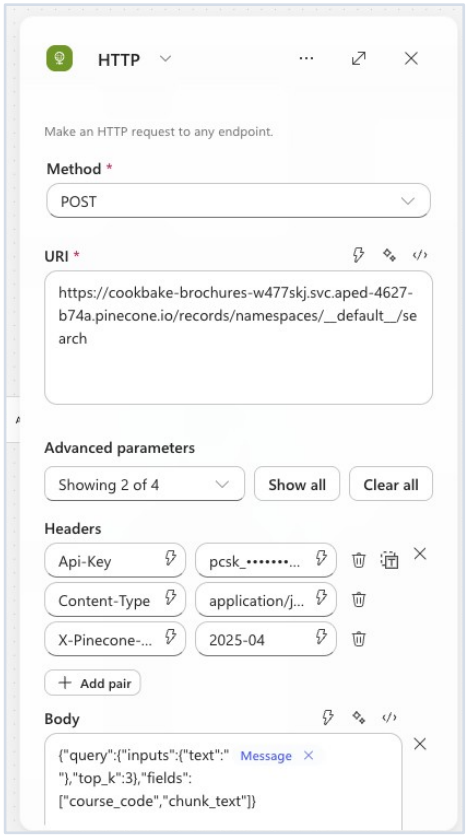


Create a NEW site — do not reuse another lab's.

The search and knowledge connectors index at folder level. Point a course assistant at a library that also holds banking policy and it will answer a sourdough question out of a KYC document.

Confirm the library shows 20 items. Nineteen means one upload silently failed — and that course becomes an "I don't have that" you will blame on the agent.

Task 2 — The HTTP Node: Retrieval



Method is a SEPARATE dropdown — do not type POST into the URI.

__default__ is literal. Pinecone's default namespace is "" in its stats but __default__ in the URL path.

Three headers. Paste the KEY, not the words PINECONE_API_KEY — Power Automate cannot read a .env file.

The key goes in Headers, never in Authentication.

Body sends the question as plain text. Pinecone embeds it server-side.

BUILDING THE PROMPT

Task 3 — The Compose Node

1. **The Agent node has NO input field**
   Only Instructions. So the per-question data has to be assembled somewhere else, and referenced from there.

2. **concat() glues three things together**
   The customer's question, a label, and the retrieved brochures.

3. **string() does the escaping**
   The HTTP response is JSON full of quotes and newlines. string() turns it into safe text. Without it the prompt breaks.

4. **It is also your debugger**
   Compose's output appears in the run history — so you can see exactly what the agent was given, which is how you diagnose an empty answer.

```
concat('Customer question: ', triggerBody()?['message'], '\n\nCourse brochures:\n', string(body('HTTP')))
```

THE SECOND HALF OF RAG

Task 4 — The Agent Node: Generation

1. **Instructions carry the rules, not the facts**
   Answer only from the brochures. Never invent a fee, date, duration, code, instructor or discount. Say so when you do not know.

2. **Then insert the Compose reference with the ⚡ picker**
   Put the cursor at the end of the instructions and pick Compose → Outputs from the dynamic content tree. A blue chip appears.

3. **NEVER paste an expression into Instructions**
   It is a rich-text editor. Pasted references are stored as plain text or have their underscores escaped.

4. **A broken reference resolves to EMPTY, not an error**
   The node stays green, the run succeeds, and the agent silently receives nothing at all.

Symptom: the agent replies “the course brochures weren't included in your message”. It is telling you exactly what is wrong, in the one place nobody looks.

A reference to nothing resolves to empty — not an error.

Green node. Successful run. No answer. This one failure mode costs more debugging time than every other mistake in this lab combined.

GETTING THE ANSWER BACK TO THE BROWSER

Task 5 — The Response Node

1. **Status code 200, body { "reply": … }**
   The chat widget reads data.reply. A different shape shows [object Object] in the bubble.

2. **The field is called Agent Response — not text**
   Use the ⚡ picker. Typing body/text yields an empty string with no error, and the flow looks broken for reasons unrelated to RAG.

3. **Then PUBLISH — not just Save**
   The HTTP endpoint serves the PUBLISHED version. Check □ → Version history: LIVE and CURRENT DRAFT must match.

4. **Copy the HTTP POST URL from the trigger**
   It ends with sig=… — if that is missing, the URL was truncated on copy and every call will fail.

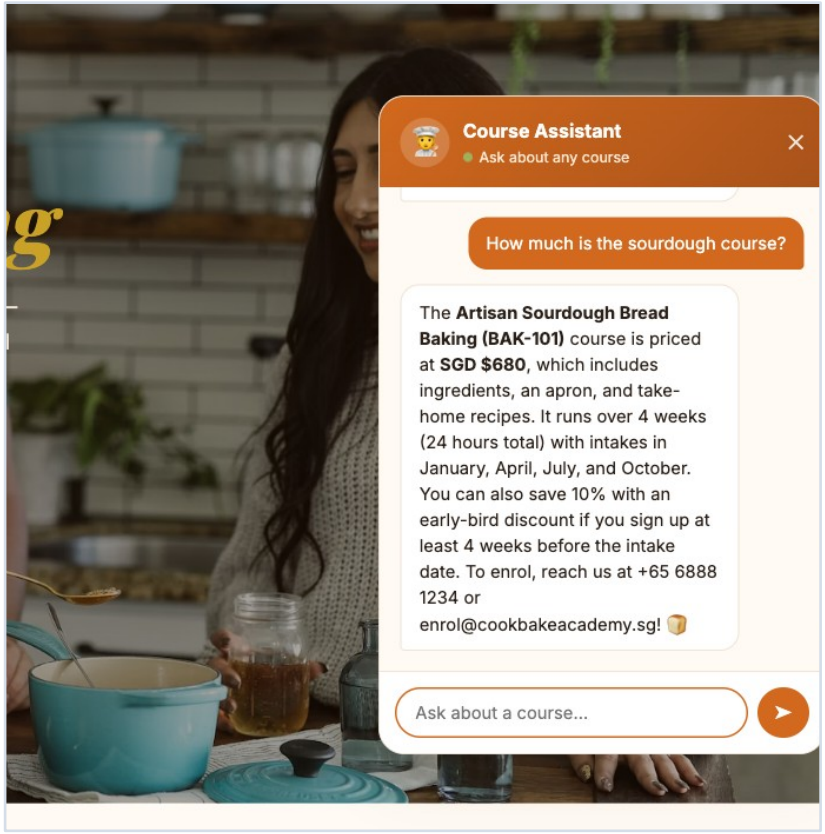
04

Testing

Try to make it lie

TC1

It Works — A Grounded Answer



Course code BAK-101. Fee SGD $680. Duration 4 weeks. Intakes Jan, Apr, Jul, Oct. The 10% early-bird discount. The enrolment phone and email.

Every one of those facts is in the brochure. None came from the model's own training.

Round trip: about 15 seconds. Set expectations in class — this is not a database lookup.

SAMPLE-QUESTIONS.CSV

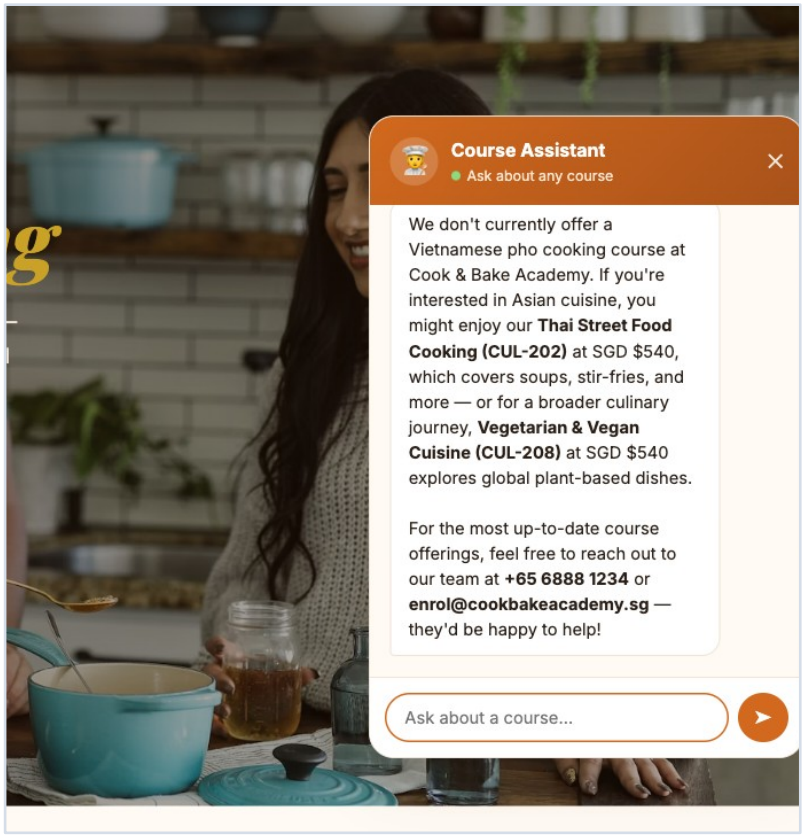
Ten Test Cases

CASE	QUESTION	A GOOD ANSWER
TC1	How much is the sourdough course?	BAK-101 + exact fee
TC2	How long is the French Pastry course?	BAK-102 + duration
TC3	Where are your campuses?	Both, with addresses
TC4	Cooking courses for beginners?	Two or three, not twenty
TC5	What is CUL-203 about?	Sushi & Sashimi + curriculum
TC6	Cheaper — macarons or cookies?	Both fees + which
TC7	Vietnamese pho course?	WE DON'T RUN THAT
TC8	Who teaches the macaron class?	NOT IN OUR INFORMATION
TC9	The 40% alumni discount on sushi?	DOES NOT CONFIRM IT
TC10	Recommend a restaurant?	Declines, steers back

The first six prove retrieval works. The last four are the ones that matter.

THE SLIDE THAT JUSTIFIES THE LAB

TC7 — The Agent Refuses to Invent



Asked for a course the academy does not run.

It says so plainly — then offers CUL-202 Thai Street Food and CUL-208 Vegetarian & Vegan, with their real fees.

An ungrounded model would have invented a pho course, a fee and a schedule. Fluently. And the customer would have quoted it back to you.

Retrieval alone would not have produced this. The instruction “if we do not run a course, say so plainly” is what turns a weak match into an honest refusal.

ADVERSARIAL TESTING

The Three Probes, and Why They Are Different

TC7 — a thing that does not exist

Invites the model to invent a course. Easy to resist, because nothing in the brochures resembles it.

TC8 — a fact in no brochure

Instructors are not in the course information. The model must distinguish “not retrieved” from “does not exist”.

TC9 — a false premise

The nastiest. The question ASSUMES a 40% discount exists. A model that wants to be helpful confirms it.

Any confident answer to these is a failure, however fluent. If yours invents an instructor, do not add instructors to the brochures — fix the instruction, then ask what else it might invent.

WHY RETRIEVAL ALONE IS NOT ENOUGH

Grounding: The Four Lines That Do the Work

What retrieval gives you

The three most similar brochures.

That is all. It is a search engine, not a conscience.

It always returns something — even when nothing fits.

A weak match still looks like a match.

What the instruction must add

Answer ONLY from the brochures below.

If they do not answer it, say “I don't have that”.

Never invent a fee, date, duration, code, instructor or discount.

If we do not run a course, say so, then list the closest.

Retrieval supplies the facts. The instruction forbids everything else. Ship one without the other and you have a confident liar or a useless one.

05

Debrief

What you built, and what it cost

THE PRICE OF CONVENIENCE

What the Platform Chose For You

1. **Embedding model — llama-text-embed-v2**
   You never picked one. Change it and every stored vector becomes meaningless until you re-ingest.

2. **Dimension — 1024**
   Fixed when the index was created. You cannot change it; you create a new index.

3. **Chunking — one brochure, one record**
   The single most consequential setting in the system, decided at upload time, invisible afterwards.

4. **Retrieved count — top_k: 3**
   The one knob you did turn. Set it to 1 and comparisons fail; set it to 20 and you are back to pasting the catalogue.

This is a genuine feature and a genuine cost: you cannot tune what you cannot see.

DEBRIEF

Five Questions for the Room

CASE	QUESTION	A GOOD ANSWER
1	The bot said “I don't have that” for TC8	**Good answer or bad?**
2	top_k is 3 — try 1, then 20	**Which failure is worse?**
3	Change a fee and re-ingest	**Who owns accuracy now?**
4	LU1 matched an NRIC exactly; this matches meaning	**When do you choose which?**
5	You never chose an embedding model	**When does that stop being OK?**

Question 3 is the one that changes how a team organises itself: the person who maintains the brochures now owns what the chatbot says.

HARD-WON, IN THIS ORDER

The Three Failures Worth Remembering

A reference that resolves to empty

Pasted into the rich-text Instructions box. Green node, successful run, no answer. Insert with the ⚡ picker, never paste.

The wrong output field

Agent Response, not text. Guessing gives you an empty string and no error to explain it.

The key that was a variable name

PINECONE_API_KEY pasted as the header VALUE. Power Automate cannot read a .env file. Unauthorized.

All three fail quietly. None of them is about RAG — and together they cost more time than the retrieval, the embedding and the grounding combined.

RAG changes the prompt, not the model.

Retrieve the few documents that answer the question. Instruct the model to use nothing else. Then spend your testing time trying to make it lie.